🧪 Práctica integradora de vectores — 2

Estación de carga de autoelevadores

Cátedra: Programación en Computación — UTN FRRQ **Docentes:** Longhi Pablo, Talijancic Iván

> 🎯 **Formato del 1er parcial**, un escalón más que la práctica 1. Hay dos cosas nuevas, y las dos aparecen en parciales: **generar un vector sin valores repetidos**, y usar un vector que guarda **posiciones de otros vectores**.
>
> **No hace falta saber de baterías.** Todo lo que hay que calcular está definido con su fórmula.

📘 Contexto

Un depósito tiene **8 autoelevadores eléctricos**. Al terminar el turno llegan a la estación de carga, que tiene **un solo cargador de 12 kW**: los atiende **de a uno, en el orden en que llegaron**, y cada uno se carga hasta el 100 %.

Los datos fijos son dos **vectores paralelos**: la posición `i` de cada uno habla del mismo autoelevador.

ÍNDICE	0	1	2	3	4	5	6	7
Autoelevador	AE-01	AE-02	AE-03	AE-04	AE-05	AE-06	AE-07	AE-08
Capacidad de la batería (kWh)	24	24	30	30	36	36	48	48

```
const FORKLIFTS = ['AE-01', 'AE-02', 'AE-03', 'AE-04',
                   'AE-05', 'AE-06', 'AE-07', 'AE-08']
const CAPACITY = [24, 24, 30, 30, 36, 36, 48, 48]   // kWh
const CHARGER_POWER = 12                            // kW
```

Se pide

1. Generación de datos

Generá dos vectores:

- **CHARGE**: la carga de cada autoelevador al llegar, en %, con enteros aleatorios entre **5 y 95**. `CHARGE[i]` es la carga del autoelevador `i`.

- **ORDER**: el **orden de llegada**. Contiene los índices **0 a 7, cada uno una sola vez**, en orden aleatorio. `ORDER[0]` es el índice del primero que llegó; `ORDER[1]`, el del segundo.

```
const getRandomVector = (size, min, max) => { ... }
const contains = (vector, value) => { ... }      // true si value está en vector
const getRandomOrder = (size) => { ... }
```

> 💡 **Cómo generar sin repetir:** para cada lugar del orden, sorteá un índice con `rndInt(0, 7)` y **volvé a sortear mientras ya esté** en el vector. Es un `do-while` con `contains()` en la condición.
>
> ⚠️ **Inicializá ORDER con -1, no con 0.** Si lo llenás con ceros, el 0 figura como «ya usado» desde el principio: cuando falte sortear el último lugar no va a quedar ningún número libre, y el programa **se cuelga sin dar ningún error**. El -1 es un índice imposible: nunca se confunde con uno real.

Para comparar con los ejemplos, mientras desarrollás usá estos vectores fijos:

```
const CHARGE = [15, 62, 40, 8, 75, 33, 50, 22]
const ORDER = [1, 6, 3, 2, 7, 0, 5, 4]
```

Ejemplo de salida:

```
Carga al llegar (%): [
  15, 62, 40,  8,
  75, 33, 50, 22
]
Orden de llegada:   [
  1, 6, 3, 2,
  7, 0, 5, 4
]
```

2. Energía y tiempo de carga

Calculá cuánta energía le falta a cada autoelevador para llegar al 100 %, y cuánto tarda en cargarla:

```
ENERGY[i] = CAPACITY[i] * (100 - CHARGE[i]) / 100        (kWh, redondeado a 1 decimal)
TIME[i]   = ENERGY[i] / 12                                (h, redondeado a 2 decimales)

energía total = ENERGY[0] + ENERGY[1] + ... + ENERGY[7]
```

```
const getEnergyVector = (capacity, charge) => { ... }
const getTimeVector = (energy, power) => { ... }
```

Ejemplo de salida:

```
Energía a cargar (kWh): [
  20.4,  9.1,   18,
  27.6,    9, 24.1,
    24, 37.4
]
Tiempo de carga (h):    [
  1.7, 0.76,  1.5,
  2.3, 0.75, 2.01,
    2, 3.12
]
Energía total: 169.6 kWh
```

3. Estado de las baterías

Clasificá cada batería según su carga al llegar, y contá cuántas hay en cada estado:

```
si CHARGE[i] < 20:              "CRITICA"
si 20 <= CHARGE[i] <= 50:       "BAJA"
si CHARGE[i] > 50:              "OK"
```

```
const levelOf = (charge) => { ... }
const getLevelVector = (charge) => { ... }
const countEqual = (vector, value) => { ... }
```

Ejemplo de salida:

```
Estado de las baterías: [
  'CRITICA', 'OK',
  'BAJA',    'CRITICA',
  'OK',      'BAJA',
  'BAJA',    'BAJA'
]
  CRITICA: 2
  BAJA:    4
  OK:      2
```

4. Cola de carga

El cargador atiende en el orden de **ORDER**: en el **turno k** carga al autoelevador `ORDER[k]`. Cada turno empieza cuando termina el anterior:

```
START[0] = 0
START[k] = START[k - 1] + TIME[ORDER[k - 1]]       (desde k = 1, redondeado a 2 decimales)
fin del turno k = START[k] + TIME[ORDER[k]]
```

Generá el vector **START** con la hora de inicio de cada turno, y mostrá la tabla de turnos con el **nombre** de cada autoelevador.

```
const getStartVector = (order, time) => { ... }
```

Ejemplo de salida:

```
Cola de carga:
  Turno 1: AE-02  de 0.00 h a 0.76 h
  Turno 2: AE-07  de 0.76 h a 2.76 h
  Turno 3: AE-04  de 2.76 h a 5.06 h
  Turno 4: AE-03  de 5.06 h a 6.56 h
  Turno 5: AE-08  de 6.56 h a 9.68 h
  Turno 6: AE-01  de 9.68 h a 11.38 h
  Turno 7: AE-06  de 11.38 h a 13.39 h
  Turno 8: AE-05  de 13.39 h a 14.14 h
```

> ⚠️ Fijate que el turno 1 es el **AE-02**, no el AE-01: el turno `k` se lee en `ORDER[k]`, y con ese índice vas a `FORKLIFTS`, `TIME` y los demás. Si tu tabla sale AE-01, AE-02, AE-03..., estás recorriendo los autoelevadores en vez de los turnos.

5. Reporte de la estación

Mostrá:

- El **tiempo total de carga**: el fin del último turno.
- El autoelevador que necesitó **más energía**: su nombre y cuánta.
- Cuántos autoelevadores en estado **CRITICA** tuvieron que **esperar más de 2 h** para empezar a cargar, es decir, con `START[k] > 2`. Si no hubo ninguno, un mensaje que lo diga.

```
const maxPosition = (vector) => { ... }        // posición del máximo
const countLateCritical = (order, start, levels, maxWait) => { ... }
```

Ejemplo de salida:

```
── Reporte de la estación ──
Tiempo total de carga: 14.14 h
Más energía: AE-08 (37.4 kWh)
Críticos que esperaron más de 2 h: 2
```

> 🤔 **Para pensar, sin programar:** dos autoelevadores casi descargados esperaron horas mientras se cargaban otros que llegaron con más del 50 %. ¿En qué orden convendría atenderlos?

⛔ Restricciones

- ❌ **Sin** `map`, `filter`, `reduce`, `forEach`, `find`, `sort`, `indexOf`, `includes`, `push`, `fill`.

- ❌ **Sin** retornar objetos: sólo números, textos, booleanos y vectores.
- ✅ Todo en **funciones** con parámetros y valor de retorno, con su **JSDoc**.
- ✅ Toda función que devuelve un vector lo devuelve con `.slice()`.
- ✅ Los vectores se crean con `new Array(n)` y se llenan con un bucle.

> 🔍 Probá cada función en la consola de Node antes de usarla: `contains([3, 1], 1)`, `contains([3, 1], 0)`, `levelOf(20)`, `levelOf(50)`.

📎 Material de consulta

- Práctica integradora 1 (https://github.com/italijancic/pc-2026/blob/main/unidades/07-arrays-unidimensionales/practica-1.pdf)
- Presentación de la unidad 07 (https://github.com/italijancic/pc-2026/blob/main/unidades/07-arrays-unidimensionales/presentacion.pdf)
- Apunte de la unidad 07 (https://github.com/italijancic/pc-2026/blob/main/unidades/07-arrays-unidimensionales/apunte.pdf)